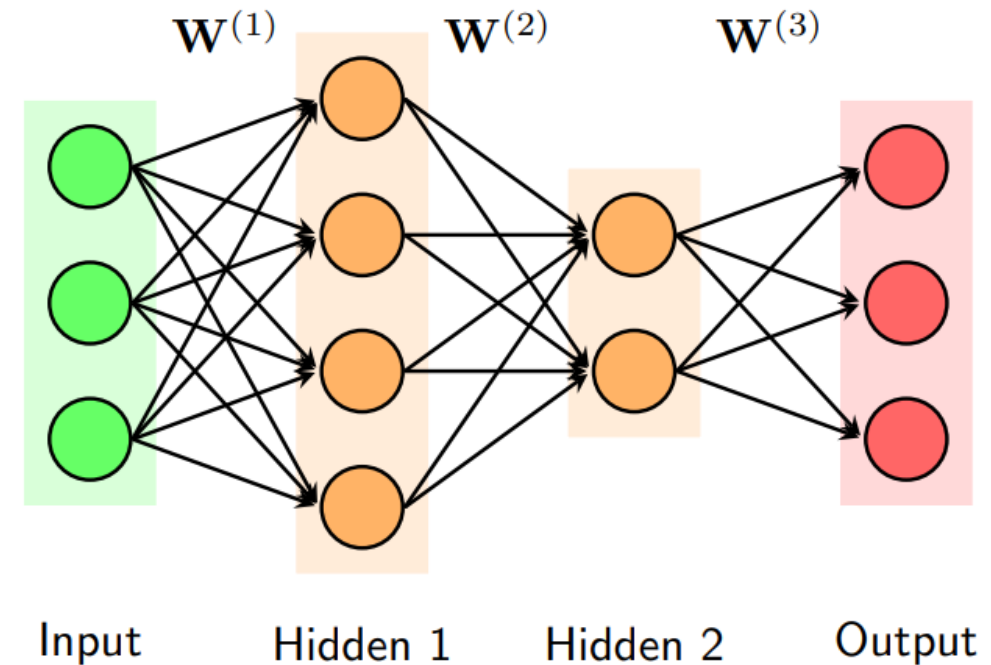


COMPUTATON IN DEEP LEARNING NETWORKS

Dr. Nguyen Thi Kim Ngan

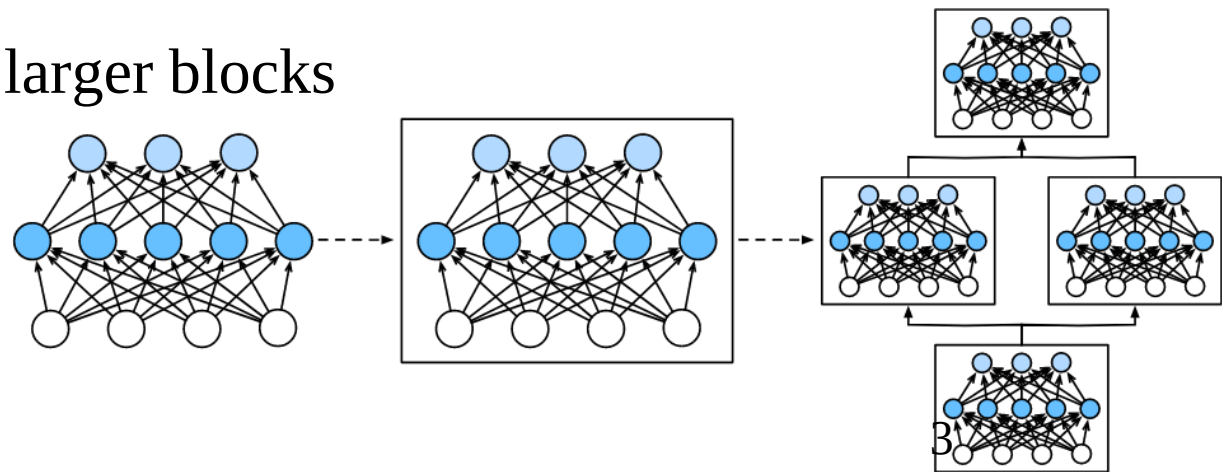
Layer

- A layer is a collection of neurons arranged vertically
- Three main types of layers:
 - Input layer
 - Hidden layer(s)
 - Output layer
- The total number of layers equals the number of hidden layers plus one
- The number of layers in a Multilayer Perceptron (MLP) is denoted by L . In the figure, $L = 3$



Block

- A block can be a single layer, a multi-layer module, or the entire model
- A block includes:
 - Input
 - Output
 - Necessary parameters (some blocks may not require any parameters)
 - A forward method to transform the Input into the Output
 - A backward method to compute gradients
- Blocks can be combined to form larger blocks





Example

- Use Dense to declare each layer
- Use Sequential to declare each block
- Create a network consisting of:
 - A fully connected hidden layer with 256 nodes, using the ReLU activation function
 - A fully connected output layer with 10 nodes (without an activation function)

```
from mxnet import np, npx
from mxnet.gluon import nn
npx.set_np()

x = np.random.uniform(size=(2, 20))

net = nn.Sequential()
net.add(nn.Dense(256, activation='relu'))
net.add(nn.Dense(10))
net.initialize()
net(x)
```



Custom block

```
from mxnet.gluon import nn
```

```
#Definition of a block class
```

```
class MLP(nn.Block):
```

```
    # Declare a layer with model parameters. Here, we declare  
    two fully connected layers
```

```
    def __init__(self, **kwargs):
```

```
        super(MLP, self).__init__(**kwargs)
```

```
        #Hidden layer
```

```
        self.hidden = nn.Dense(256, activation='relu')
```

```
        # Output layer
```

```
        self.output = nn.Dense(10)
```

```
    def forward(self, x):
```

```
        return self.output(self.hidden(x))
```

```
#Using a instance block
```

```
net = MLP()
```

```
net.initialize()
```

```
net(x)
```



Sequential blocks

```
From mxnet.gluon import nn
class MySequential(nn.Block):
    def add(self, block):
        self._children[block.name] = block

    def forward(self, x):
        for block in self._children.values():
            x = block(x)
        return x
```

```
net = MySequential()
net.add(nn.Dense(256, activation='relu'))
net.add(nn.Dense(10))
net.initialize()
net(x)
```



Parallel blocks

```
from mxnet import nd
from mxnet.gluon import nn, Block
```

```
#Defining parallel blocks
```

```
class ComplexBlock(Block):
```

```
    def __init__(self, **kwargs):
```

```
        super(ComplexBlock, self).__init__(**kwargs)
```

```
        with self.name_scope():
```

```
            self.blockA = nn.Dense(16)    # Block A
```

```
            self.blockB = nn.Dense(16)    # Block B
```

```
            self.blockC = nn.Dense(8)     # Block C (following the concatenation of A and B)
```

```
    def forward(self, x):
```

```
        outA = self.blockA(x)
```

```
        outB = self.blockB(x)
```

```
#The outputs of A and B are concatenated along the feature axis
```

```
        concat = nd.concat(outA, outB, dim=1)
```

```
        outC = self.blockC(concat)
```

```
        return outC
```

```
net = ComplexBlock()
```

```
net.initialize()
```

```
x = nd.random.uniform(shape=(4, 10)) # batch 4, input dim=10
```

```
y = net(x)
```

```
print("Input shape:", x.shape)
```

```
print("Output shape:", y.shape)
```



Composing blocks

```
from mxnet.gluon import nn
```

```
class NestMLP(nn.Block):
```

```
    def __init__(self, **kwargs):
        super(NestMLP, self).__init__(**kwargs)
        self.net = nn.Sequential()
        self.net.add(nn.Dense(64, activation='relu'),
                     nn.Dense(32, activation='relu'))
        self.dense = nn.Dense(16, activation='relu')
```

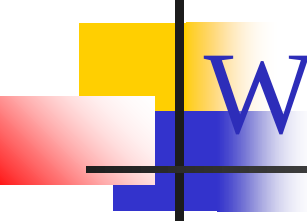
```
    def forward(self, x):
        return self.dense(self.net(x))
```

```
chimera = nn.Sequential()
```

```
chimera.add(NestMLP(), nn.Dense(20), MLP())
```

```
chimera.initialize()
```

```
chimera(x)
```



Weight and bias management

- For example, a Perceptron network with one hidden layer

```
from mxnet import init, np, npx
```

```
from mxnet.gluon import nn
```

```
npx.set_np()
```

```
net = nn.Sequential()
```

```
net.add(nn.Dense(256, activation='relu'))
```

```
net.add(nn.Dense(10))
```

```
net.initialize() # Use the default initialization method
```

```
x = np.random.uniform(size=(2, 20))
```

```
net(x) # Forward computation
```

#Accessing model parameters at each layer

```
net[0].params
```

```
net[1].params
```

#Accessing parameters via the weight and bias attribute

```
net[1].bias
```

```
net[1].bias.data()
```

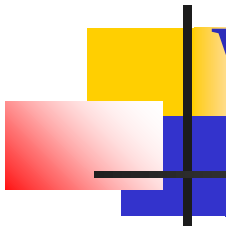
#Accessing parameters by their names

```
net[0].params['dense0_weight']
```

```
net[0].params['dense0_weight'].data()
```

#Accessing gradients

```
net[0].weight.grad()
```



Weight and bias management

- For example, a Perceptron network with one hidden layer

```
from mxnet import init, np, npz
```

```
from mxnet.gluon import nn
```

```
npz.set_np()
```

```
net = nn.Sequential()
```

```
net.add(nn.Dense(256, activation='relu'))
```

```
net.add(nn.Dense(10))
```

```
net.initialize() # Use the default initialization method
```

```
x = np.random.uniform(size=(2, 20))
```

```
net(x) # Forward computation
```

#Accessing the weights and bias of the first layer

```
net[0].collect_params()
```

Accessing all weights and biases of net

```
net.collect_params()
```

#Accessing the weights and bias by name

```
net.collect_params('.*weight')
```



Weights and biases from nested blocks

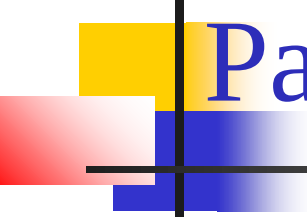
```
def block1():  
    net = nn.Sequential()  
    net.add(nn.Dense(32, activation='relu'))  
    net.add(nn.Dense(16, activation='relu'))  
    return net
```

```
def block2():  
    net = nn.Sequential()  
    for i in range(4):  
        net.add(block1())  
    return net
```

```
rgnet = nn.Sequential()  
rgnet.add(block2())  
rgnet.add(nn.Dense(10))  
rgnet.initialize()  
rgnet(x)
```

```
#Collecting all parameters (weights and biases)  
rgnet.collect_params  
rgnet.collect_params()
```

```
# Accessing a specific parameter  
rgnet[0][1][0].bias.data()
```



Parameter initialization

- All parameters were initialized from a Gaussian distribution with a standard deviation of 0.01
`net.initialize(init=init.Normal(sigma=0.01), force_reinit=True)`
`net[0].weight.data()[0]`
- All parameters are initialized to a constant value using the Constant(1) initializer
`net.initialize(init=init.Constant(1), force_reinit=True)` `net[0].weight.data()[0]`
- Initializing parameters for different blocks
 - `net[0].weight.initialize(init=init.Xavier(), force_reinit=True)`
`net[1].initialize(init=init.Constant(42), force_reinit=True)`
`net[1].weight.data()[0, 0]`



Parameter initialization

```
class MyInit(init.Initializer):  
    def _init_weight(self, name, data):  
        print('Init', name, data.shape)  
        data[:] = np.random.uniform(-10, 10, data.shape)  
        data *= np.abs(data) >= 5
```

```
net.initialize(MyInit(), force_reinit=True)  
net[0].weight.data()[0]
```



Parameter initialization

```
net[0].weight.data()[:] += 1
```

```
net[0].weight.data()[0, 0] = 42
```

```
net[0].weight.data()[0]
```